STRING

GAME PROGRAMMING COURSE — C# BASICS

文字列（string）を使いこなす

1 文字ずつ見る・切り取る・組み立てる

対象：1 年生（クラスまで既習）

LESSON 1

文字列の基本

1

目標：文字列が「文字の並び」だと分かり、１文字ずつ取り出せる

文字列の基本

string と char：文字列は「文字の並び」

C# Program.cs

```
1  string name = "paiza";
2  char first = 'p';
3  Console.WriteLine(name);
4  Console.WriteLine(first);
```

出力

```
paiza
p
```

型	入れるもの	書き方
string	文字列（0 文字以上）	"paiza"　ダブルクォート
char	1 文字だけ	'p'　シングルクォート

- string の中身は char が 1 列に並んだもの
- 1 文字取り出すと char になる



POINT　" " は string、' ' は char。1 文字でも "p" と 'p' は別の型。

文字列の基本

文字数と、i 文字目の取り出し

C# Program.cs

```
1  string text = Console.ReadLine();
2  Console.WriteLine(text.Length);
3  Console.WriteLine(text[0]);
4  Console.WriteLine(text[text.Length - 1]);
```

入力例

```
paiza
```

出力

```
5
p
a
```

やりたいこと	書き方
文字数	text.Length
最初の文字	text[0]
最後の文字	text[text.Length - 1]
Paiza の「i 文字目」	text[i - 1]

注意 番号は配列と同じで 0 から。問題文の「i 文字目」は 1 から数えているので、必ず i - 1 する。

文字列の基本

for で 1 文字ずつ見る

番号（何文字目か）も使う

C# Program.cs

```
1  string text = "abc";
2  for (int i = 0; i < text.Length; i++)
3  {
4      Console.WriteLine(i + ":" + text[i]);
5  }
```

出力

```
0:a
1:b
2:c
```

1 文字を変数 c に入れて使う

C# Program.cs

```
1  string text = "abc";
2  for (int i = 0; i < text.Length; i++)
3  {
4      char c = text[i];
5      Console.WriteLine(c);
6  }
```

出力

```
a
b
c
```

POINT 文字列は配列とほぼ同じ。配列 numbers[i] → 文字列 text[i]、numbers.Length → text.Length に置きかえるだけ。取り出した 1 文字は char。

文字列の基本

文字列をつなげる（連結）

C# Program.cs

```
1  string first = "pai";
2  string second = "za";
3  string word = first + second;
4  Console.WriteLine(word);
5  string result = "";
6  result += "A";
7  result += "B";
8  Console.WriteLine(result);
9  Console.WriteLine("HP:" + 10 + 5);
10 Console.WriteLine("HP:" + (10 + 5));
```

出力

```
paiza
AB
HP:105
HP:15
```

- + で文字列どうしをつなげる
- result = ""（空の文字列）から += で 1 つずつ足していける
- **文字列 + 数値 は「文字としてつながる」**

注意 "HP:" + 10 + 5 は左から順に「HP:10」→「HP:105」。計算したいときは ( ) で先に計算させる。

文字列の基本

文字列・文字の比べ方

C# Program.cs

```
1  string answer = Console.ReadLine();
2  if (answer == "yes")
3  {
4      Console.WriteLine("OK");
5  }
6  char first = answer[0];
7  if (first == 'y')
8  {
9      Console.WriteLine("y から始まる");
10 }
```

入力 yes → 出力

```
OK
y から始まる
```

比べるもの	書き方
string と string	answer == "yes"
char と char	answer[0] == 'y'
×char と string	answer[0] == "y"（エラー）

- text[i] で取り出したものは char
- char と比べるときは ' ' を使う

文字列の基本

型の変換：文字列 ⇔ 数値 ⇔ 文字

変換	書き方	例 → 結果
string → int	int.Parse(text)	int.Parse("42") → 42
int → string	number.ToString()	42.ToString() → "42"
string → char	text[i]	"paiza"[0] → 'p'
char → string	c.ToString()	'p'.ToString() → "p"
数字の char → int	c - '0'	'7' - '0' → 7
string → char 配列	text.ToCharArray()	"abc" → { 'a', 'b', 'c' }
char 配列 → string	new string(chars)	{ 'a', 'b', 'c' } → "abc"

注意 int.Parse は「数字だけの文字列」しか変換できない。"12 34" のようにスペースが入っていたら、先に Split する。

LESSON 2

2

文字列の便利な機能

目標：Paiza でよく使う文字列のメソッドを使い分けられる

まずは全体

よく使うメソッドを一度に試すプログラム

C# Program.cs

```
1  string text = Console.ReadLine();
2  Console.WriteLine(text.Length);
3  Console.WriteLine(text.ToUpper());
4  Console.WriteLine(text.ToLower());
5  Console.WriteLine(text.Substring(6, 5));
6  Console.WriteLine(text.Contains("Pai"));
7  Console.WriteLine(text.IndexOf("Pai"));
8  Console.WriteLine(text.Replace("Hello", "Bye"));
```

入力例

```
Hello Paiza
```

出力

```
11
HELLO PAIZA
hello paiza
Paiza
True
6
Bye Paiza
```

POINT このあと、1行ずつ「何をしているか」を分けて見ていく。

2〜4 行目

文字数・大文字・小文字

文字列ぜんぶに使う

書き方	結果（text = "Paiza"）
text.Length	5
text.ToUpper()	"PAIZA"
text.ToLower()	"paiza"

1 文字（char）に使う

書き方	意味	例 → 結果
char.IsUpper(c)	大文字？	'P' → True
char.IsLower(c)	小文字？	'a' → True
char.IsDigit(c)	数字？	'7' → True
char.ToUpper(c)	大文字にする	'a' → 'A'
char.ToLower(c)	小文字にする	'P' → 'p'

POINT Is〜 は true / false を返すので、そのまま if の条件に入れられる。例：if (char.IsUpper(c)) { … }

5 行目

Substring：文字列の一部を切り取る



書き方	意味
text.Substring(開始, 文字数)	開始の番号から、文字数ぶん切り取る
text.Substring(開始)	開始の番号から、最後まで

text.Substring(6, 5)

"Paiza"（[6] から 5 文字）

Paiza の「i 文字目から j 文字目」

```
text.Substring(i - 1, j - i + 1)
```

注意 2 つ目は「終わりの番号」ではなく「文字数」。ここを間違える人がとても多い。

6〜7 行目

Contains / IndexOf：文字列を探す

C# Program.cs

```
1  string text = "paiza";
2  Console.WriteLine(text.Contains("iz"));
3  Console.WriteLine(text.IndexOf('z'));
4  Console.WriteLine(text.IndexOf('x'));
5  Console.WriteLine(text.IndexOf('z') + 1);
```

出力

```
True
3
-1
4
```

書き方	返すもの
Contains("iz")	含まれていれば True
IndexOf('z')	最初に見つかった番号
見つからないとき	IndexOf は -1

- 「何文字目にあるか」は IndexOf + 1
- Contains は if の条件にそのまま使える

8 行目と書き換え

Replace と、1 文字だけ書き換える方法

Replace：同じ文字を全部置きかえ

C# Program.cs

```
1  string text = "paiza";
2  Console.WriteLine(text.Replace("a", "A"));
```

出力

```
pAizA
```

✕ text[0] = 'P'; はエラー

文字列は作ったあと、中の 1 文字を直接書き換えられない

○ char 配列にしてから書き換える

C# Program.cs

```
1  string text = "paiza";
2  char[] chars = text.ToCharArray();
3  chars[0] = 'P';
4  string result = new string(chars);
5  Console.WriteLine(result);
```

出力

```
Paiza
```

POINT　「i 文字目だけ変える」→ ToCharArray → chars[i - 1] = 新しい文字 → new string(chars) の 3 ステップ。

復習＋α

Split / string.Join / new string

C# Program.cs

```
1  string text = "red green blue";
2  string[] colors = text.Split(" ");
3  Console.WriteLine(colors[1]);
4  Console.WriteLine(string.Join(",", colors));
5  Console.WriteLine(new string('*', 5));
```

出力

```
green
red,green,blue
*****
```

書き方	意味
text.Split(" ")	区切って配列にする
string.Join(",", 配列)	配列を , でつないで 1 つの文字列に
new string('*', 5)	同じ文字を 5 個並べる

POINT string.Join(" ", numbers) で、配列をスペース区切りの 1 行で出力できる。Paiza の出力でよく使う。

一覧

文字列メソッド 一覧（演習中に見返そう）

書き方	意味	例（text = "paiza"）→ 結果
text.Length	文字数	5
text[i]	i 番目の文字（char）	text[1] → 'a'
text.ToUpper() / ToLower()	大文字 / 小文字にする	"PAIZA"
text.Substring(開始, 文字数)	一部を切り取る	Substring(1, 3) → "aiz"
text.Contains(文字列)	含まれているか	Contains("iz") → True
text.IndexOf(文字)	何番目にあるか（なければ -1）	IndexOf('z') → 3
text.Replace(前, 後)	全部置きかえる	Replace("a", "A") → "pAizA"
text.Split(" ")	区切って配列にする	"a b" → { "a", "b" }
string.Join(" ", 配列)	配列をつないで文字列に	{ "a", "b" } → "a b"
text.ToCharArray()	char 配列にする（書き換え用）	{ 'p', 'a', 'i', 'z', 'a' }

LESSON 3

3

Paiza の例題に挑戦

目標：文字列の機能と型を選んで、例題を自分の力で解く

例題 STEP 1

文字列の基本操作

i 文字目の出力 `text[i - 1]`

https://paiza.jp/works/mondai/string_primer/basic_step3/edit?language_uid=c-sharp

文字列の文字数 `Length`

https://paiza.jp/works/mondai/string_primer/basic_step5/edit?language_uid=c-sharp

部分文字列 `Substring`

https://paiza.jp/works/mondai/string_primer/normal_step1/edit?language_uid=c-sharp

文字列の検索 `Contains`

https://paiza.jp/works/mondai/string_primer/normal_step9/edit?language_uid=c-sharp

POINT どれも for はいらない。「どのメソッドを使うか」を一覧から選ぼう。

例題 STEP 2

for で 1 文字ずつ・つなげて作る

文字列の連結 型④ 文字列版

https://paiza.jp/works/mondai/string_primer/basic_step7/edit?language_uid=c-sharp

文字の検索 IndexOf / 型⑦

https://paiza.jp/works/mondai/string_primer/basic_step6/edit?language_uid=c-sharp

大文字小文字の反転 型① + if

https://paiza.jp/works/mondai/string_primer/normal_step8/edit?language_uid=c-sharp

文字列の反転 後ろからの for

https://paiza.jp/works/mondai/string_primer/normal_step10/edit?language_uid=c-sharp

POINT 「result = "" から += でつなげる」形が何度も出てくる。手が止まったら if / for 授業の「型」を見返そう。

例題 STEP 3

機能と型を組み合わせる

文字列の書き換え ToCharArray

https://paiza.jp/works/mondai/string_primer/normal_step3/edit?language_uid=c-sharp

回文判定 反転 + 比較

https://paiza.jp/works/mondai/string_primer/normal_step11/edit?language_uid=c-sharp

数値判定 型⑥ すべて〜か

https://paiza.jp/works/mondai/string_primer/advance_step4/edit?language_uid=c-sharp

重複の削除 Contains + 連結

https://paiza.jp/works/mondai/string_primer/advance_step5/edit?language_uid=c-sharp

POINT 早く終わった人は、STEP 2 の問題を「別の解き方」でもう一度解いてみよう（例：反転を for で 1 文字ずつ比べて回文判定）。

授業課題（兼 宿題）

スキルチェックに挑戦

- 授業内で取り組み、時間内に終わらなかった問題は宿題とします

C154：半額クーポン
https://paiza.jp/challenges/760/ready

C143：ファイル名の統一
https://paiza.jp/challenges/706/ready

C117：大量出店
https://paiza.jp/challenges/571/ready

POINT スキルチェックは自分の力で解くこと。困ったら「文字列メソッド 一覧」と Paiza 辞書を引こう。